Introduction to C
(C Programming Language)

Dr. Thien Huynh-The

Dept. Comp. Commun. Eng.

HCMC Technology and Education

Content

- Introduction to C Programming Language
- A simple C Program: Printing a Line of text
- Another simple C Program: adding two integers
- Memory concepts
- Arithmetic in C
- Decision making: equality and relational operators

C programming language

- Structured and disciplined approach to program design.

```
1  /* Fig. 2.1: fig02_01.c
2     A first program in C */
3  #include <stdio.h>
4
5  /* function main begins program execution */
6  int main( void )
7  {
8     printf( "Welcome to C!\n" );
9
10    return 0; /* indicate that program ended successfully */
11
12 } /* end function main */
```

/*...*/ indicates comments – ignored by compiler

#include allows C to load a particular file

A part of every C program. The parentheses after main indicate that main is a program building block called a function

Statement tells C to perform an action

return statement ends the function

Right brace declares and of main function

```
Welcome to C!
```

Line 7: “{ “ - Left brace declares beginning of main function

Simple C: Print a line of text

- Comments:
  - Text surrounded by /*...*/ is ignored by C compiler
  - Used to described program or give a note/guidance for later.

```
#include <stdio.h>
```

  - Preprocessor directive to tell C that the propgram needs to load contents of a certain file.
  - `<stdio.h>` allows standard input/output operations.
- Errors:
  - Forgetting to terminate a comment with */
  - Starting a comment with */ and ending a comment with /*
  - Another solution – using // so each line of comment

Simple C: Print a line of text

- int main()
  - C++ programs contain one or more functions, exactly one of which must be main
  - Parenthesis used to indicate a function
  - Int means that the main returns an integer value
  - Braces {...} indicate a block. The bodies of all functions must be contained in braces.
- Tips:
  - Every function should have a comment describing the purpose

Simple C: Print a line of text

```
printf( “Welcome to C!\n” );
```

- Instructs computer to perform an action
  - Specifically, prints the string of characters within quotes (“...”)
- Entire line called a statement
  - All statements must end with a semicolon
- Escape character ()
  - Indicates that the function `printf` should do something out of the ordinary
  - `\n` is the newline character

Escape sequence	Description
`\n`	Newline. Position the cursor at the beginning of the next line.
`\t`	Horizontal tab. Move the cursor to the next tab stop.
`\a`	Alert. Produces a sound or visible alert without changing the current cursor position.
`\\`	Backslash. Insert a backslash character in a string.
`\"`	Double quote. Insert a double-quote character in a string.

Simple C: Print a line of text

```
return 0;
```

- A way to exit a function
- return 0, in this case, means that the program terminated normally

- Right brace }
  - Indicates end of main has been reached
- Linker
  - When a function is called, linker locates it in the library
  - Inserts it into object program
  - If function name is misspelled, the linker will produce an error because it will not be able to find function in the library

Simple C: Print a line of text

- Tips:
  - Add a comment to the line containing the right brace, }, that closes every function, including `main`.
  - The last character printed by a function that displays output should be a newline (`\n`).
  - Indent the entire body of each function one level of indentation
  - Tab key may be used to create indents.

Assignment 1 (1-1)

- Write a simple program to print `“Welcome to C!”` in a single line using multiple `printf` functions.
  - Output will be like as follows:

```
Welcome to C!
```

- Write a simple program to print `“Welcome to C!”` in multiple lines with each line containing one words using a single `printf` function
  - Output will be like as follows:

```
Welcome
to
C!
```

Assignment 2 (1-2)

- Write a program to display a sequence of text as below

```
“Department of Computer and Communications,
    Faculty of Electrics and Electronics Engineering,
    HCM City University of Technology and Education.”
```

Simple C: Adding two integers

```c
// Fig. 2.5: fig02_05.c
// Addition program.
#include <stdio.h>

// function main begins program execution
int main( void )
{
   int integer1; // first number to be entered by user
   int integer2; // second number to be entered by user

   printf( "Enter first integer\n" ); // prompt
   scanf( "%d", &integer1 ); // read an integer

   printf( "Enter second integer\n" ); // prompt
   scanf( "%d", &integer2 ); // read an integer

   int sum; // variable in which sum will be stored
   sum = integer1 + integer2; // assign total to sum

   printf( "Sum is %d\n", sum ); // print sum
} // end function main
```

Definitions of variables

Displays the literal "Enter first/second integer" and positions the cursor to the beginning of the next line.

Obtain a value from the user and assign it to the variable integer1/integer2

Calculates the total of variables integer1 and integer2 and assigns the result to variable sum

Print the literal Sum is followed by the numerical value of variable sum

```
Enter first integer
45
Enter second integer
72
Sum is 117
```

Simple C: Adding two integers

- As before
  - Comments, `#include <stdio.h>` and `main`

```
int integer1, integer2, sum;
```

  - Definition of variables
    - Variables: locations in memory where a value can be stored
  - int means the variables can hold integers (-1, 3, 0, 47)
  - Variable names (identifiers)
    - `integer1, integer2, sum`
    - Identifiers: consist of letters, digits (cannot begin with a digit) and underscores(_)
    - Case sensitive: uppercase and lowercase letters are different in C
  - Definitions appear before executable statements
    - If an executable statement references and undeclared variable it will produce a syntax (compiler) error

Simple C: Adding two integers

- Errors:
  - Typing Main instead of main.
  - Placing variable definitions among executable statements causes syntax errors.
- Tips:
  - Multiple-word variable names can help make a program more readable. `total_commissions` or `totalCommissions`.
  - Separate the definitions and executable statements in a function

Simple C: Adding two integers

```
scanf( "%d", &integer1 );
```

- Obtains a value from the user
  - `scanf` uses standard input (usually keyboard)
- This `scanf` statement has two arguments
  - `%d` - indicates data should be a decimal integer
  - `&integer1` - location in memory to store variable
  - `&` is confusing in beginning – for now, just remember to include it with the variable name in `scanf` statements
- When executing the program the user responds to the `scanf` statement by typing in a number, then pressing the enter (return) key

- Tips:
  - Place a space after each comma (`,`) to make programs more readable.

Simple C: Adding two integers

`=` (or assignment operator)

- Assigns a value to a variable
- Is a binary operator (has two operands) besides `+`
  - `sum = variable1 + variable2;`
  - sum gets value of the expression `variable1 + variable2`
- Variable receiving value on left

```
printf( "Sum is %d \ n", sum );
```

- Similar to `scanf`
  - `%d` means decimal integer will be printed
  - `sum` specifies what integer will be printed
- Calculations can be performed inside `printf` statements
  - `printf( "Sum is %d \ n", integer1 + integer2 );`
  - No need to define the sum variable

Simple C: Adding two integers

- Errors:
  - A calculation in an assignment statement must be on the right side of the `=` operator. It is a syntax error to place a calculation on the left side of an assignment operator.
  - Forgetting to precede a variable in a `scanf` statement with an ampersand `&`
  - Forgetting one or both of the double quotes surrounding the format control string in a `printf` or `scanf`.
  - Forgetting the `%` in a conversion specification in the format control string of a `printf` or `scanf`.
  - Placing an escape sequence such as `\n` outside the format control string of a `printf` or `scanf`.

Memory Concepts

- Variables
  - Variable names correspond to locations in the computer's memory
  - Every variable has a name, a type, a size and a value
  - Whenever a new value is placed into a variable (through `scanf`, for example), it replaces (and destroys) the previous value
  - Reading variables from memory does not change them

Memory Concepts

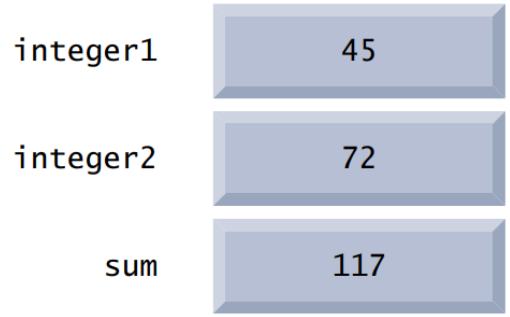
```
scanf( "%d", &integer1 );
scanf( "%d", &integer2 );
```

- Suppose that a user
  - Enter the number 45 as the value for integer1
  - Enter the number 72 as the value for integer2
- The computer will
  - Place 45 into location integer1 on memory
  - Place 72 into location integer2 on memory

Memory locations after both variables are input.

Memory Concepts

- Whenever a value is placed in a memory location, the value replaces the previous value in that location and the previous value is lost.
- `sum = integer1 + integer2;` performs the addition also replaces whatever value was stored in sum.
- When a value is read from a memory location, the process is said to be nondestructive.



Memory locations after a calculation.

Arithmetic in C

- Arithmetic calculations
  - Use `*` for multiplication and `/` for division
  - Integer division truncates remainder
    - 7 / 5 evaluates to 1
  - Modulus operator `%` returns the remainder
    - 7 % 5 evaluates to 2
- Operator precedence
  - Some arithmetic operators act before others (i.e., multiplication before addition)
    - Use parenthesis when needed
  - Example: Find the average of three variables a, b and c
    - Do not use: `a + b + c / 3`
    - Use: `(a + b + c ) / 3`

Arithmetic in C

- Errors:
  - An attempt to divide by zero is normally undefined on computer systems and generally results in a fatal error
  - Nonfatal errors allow programs to run to completion, often producing incorrect results.

C operation	Arithmetic operator	Algebraic expression	C expression
Addition	+	$f + 7$	`f + 7`
Subtraction	–	$p - c$	`p - c`
Multiplication	*	bm	`b * m`
Division	/	x / y or $\frac{x}{y}$ or $x \div y$	`x / y`
Remainder	%	$r \bmod s$	`r % s`

Fig. 2.9 | Arithmetic operators.

Arithmetic in C

- Tips:
  - Using redundant parentheses in complex arithmetic expressions can make the expressions clearer.

Operator(s)	Operation(s)	Order of evaluation (precedence)
( )	Parentheses	Evaluated first. If the parentheses are nested, the expression in the *innermost* pair is evaluated first. If there are several pairs of parentheses "on the same level" (i.e., not nested), they're evaluated left to right.
* / %	Multiplication Division Remainder	Evaluated second. If there are several, they're evaluated left to right.
+ -	Addition Subtraction	Evaluated third. If there are several, they're evaluated left to right.
=	Assignment	Evaluated last.

Fig. 2.10 | Precedence of arithmetic operators.

Arithmetic in C

Step 1.	`y = 2 * 5 * 5 + 3 * 5 + 7;` `2 * 5 is 10`	*(Leftmost multiplication)*
Step 2.	`y = 10 * 5 + 3 * 5 + 7;` `10 * 5 is 50`	*(Leftmost multiplication)*
Step 3.	`y = 50 + 3 * 5 + 7;` `3 * 5 is 15`	*(Multiplication before addition)*
Step 4.	`y = 50 + 15 + 7;` `50 + 15 is 65`	*(Leftmost addition)*
Step 5.	`y = 65 + 7;` `65 + 7 is 72`	*(Last addition)*
Step 6.	`y = 72`	*(Last operation—place* `72` *in* `y`*)*

Fig. 2.11 | Order in which a second-degree polynomial is evaluated.

Floating Point Arithmetic

- Replacing `%d` by `%f` to read a real number
- Write a program to calculate circumference with R=2

```
#include <stdio.h>
int main(void)
{
float pi;
printf( "Value of pi: \n");
scanf( "%f", &pi);
R=2;
printf( "Circumference with R=1 is %f", 2*pi*R );
return 0;
}
```

Assignment 3

- Ex1-3: Write a program to calculate $(a+b)^3$ with a and b are inputted by user.
- Ex1-4: Write a program to calculate the volume of a cylinder with radius and height inputted by user

Equality and Relational Operators

- Executable statements
  - Perform actions (calculations, input/output of data)
  - Perform decisions
    - May want to print "pass" or "fail" given the value of a test grade
- if control statement
  - Simple version in this section, more detail later
  - If a condition is true, then the body of the if statement executed
    - 0 is false, non-zero is true
  - Control always resumes after the if structure
- Keywords
  - Special words reserved for C
  - Cannot be used as identifiers or variable names

Equality and Relational Operators

Algebraic equality or relational operator	C equality or relational operator	Example of C condition	Meaning of C condition
Relational operators			
>	>	x > y	x is greater than y
<	<	x < y	x is less than y
≥	>=	x >= y	x is greater than or equal to y
≤	<=	x <= y	x is less than or equal to y
Equality operators			
=	==	x == y	x is equal to y
≠	!=	x != y	x is not equal to y

Fig. 2.12 | Equality and relational operators.

Equality and Relational Operators

- Errors:
  - If the two symbols in any of the operators ==, !=, >= and <= are separated by spaces.
  - If the two symbols in any of the operators `!=`, >= and <= are reversed as in `=!`, => and =<, respectively.
  - Confusing the equality operator == with the assignment operator =.
  - Placing a semicolon immediately to the right of the right parenthesis after the condition in an if statement.
  - Placing commas (when none are needed) between conversion specifiers in the format control string of a `scanf` statement.

Equality and Relational Operators

- Tips:
  - Indent the statement(s) in the body of an `if` statement.
  - Place a blank line before and after every `if` statement in a program for readability.
  - Although it is allowed, there should be no more than one statement per line in a program.

Equality and Relational Operators

```c
// Fig. 2.13: fig02_13.c
// Using if statements, relational
// operators, and equality operators.
#include <stdio.h>

// function main begins program execution
int main( void )
{
   printf( "Enter two integers, and I will tell you\n" );
   printf( "the relationships they satisfy: " );

   int num1; // first number to be read from user
   int num2; // second number to be read from user

   scanf( "%d %d", &num1, &num2 ); // read two integers

   if ( num1 == num2 ) {
      printf( "%d is equal to %d\n", num1, num2 );
   } // end if

```

Check that num1 is equal to num2 or not.

If they are equal together, computer executes printf

Equality and Relational Operators

```
21      if ( num1 != num2 ) {
22         printf( "%d is not equal to %d\n", num1, num2 );
23      } // end if
24
25      if ( num1 < num2 ) {
26         printf( "%d is less than %d\n", num1, num2 );
27      } // end if
28
29      if ( num1 > num2 ) {
30         printf( "%d is greater than %d\n", num1, num2 );
31      } // end if
32
33      if ( num1 <= num2 ) {
34         printf( "%d is less than or equal to %d\n", num1, num2 );
35      } // end if
36
37      if ( num1 >= num2 ) {
38         printf( "%d is greater than or equal to %d\n", num1, num2 );
39      } // end if
40   } // end function main
```

Equality and Relational Operators

- Tips:
  - No more than one statement per line.
  - Choose breaking points that make sense
  - Confirm that the operators in the expression are applied in the proper order.

Assignment 4 (1-5)

- Write a program to print the tax order and tax percentage with income entered by user using `if` statement.

Order	Income per year (M)	Tax (%)
1	Up to 60	5
2	From 60 up to 120	10
3	From 120 up to 216	15
4	From 216 up to 384	20
5	From 384 up to 624	25
6	From 624 up to 960	30
7	Over 960	35

Secure C Programming

- Instead of `printf( "Welcome to C! \ n" );`, we should write `puts( "Welcome to C!" );`
- Instead of `printf(Welcome);`, we should write `printf( "%s", "Welcome " );`
- Although the `printf` in this chapter as written are actually not insecure, these changes are responsible coding practices that will eliminate certain security vulnerabilities as we get deeper into C.

Reminder

Operators	Associativity
()	left to right
* / %	left to right
+ -	left to right
< <= > >=	left to right
== !=	left to right
=	right to left

Fig. 2.14 | Precedence and associativity of the operators discussed so far.

Reminder

Keywords				
auto	do	goto	signed	unsigned
break	double	if	sizeof	void
case	else	int	static	volatile
char	enum	long	struct	while
const	extern	register	switch	
continue	float	return	typedef	
default	for	short	union	

Keywords added in C99 standard

_Bool _Complex _Imaginary inline restrict

Keywords added in C11 standard

_Alignas _Alignof _Atomic _Generic _Noreturn _Static_assert _Thread_local

Fig. 2.15 | C's keywords.

Homeworks

E1: Write a single C statement to accomplish each of the following:

- a) Define the variables c, thisVariable, q76354 and number to be of type int.
- b) Prompt the user to enter an integer. End your prompting message with a colon (:) followed by a space and leave the cursor positioned after the space.
- c) Read an integer from the keyboard and store the value entered in integer variable a.
- d) If number is not equal to 7, print "The variable number is not equal to 7."
- e) Print the message "This is a C program." on one line.
- f) Print the message "This is a C program." on two lines so that the first line ends with C.
- g) Print the message "This is a C program." with each word on a separate line.
- h) Print the message "This is a C program." with the words separated by tabs.

Homeworks

E2: Write a statement (or comment) to accomplish each of the following:

- a) State that a program will calculate the product of three integers.
- b) Prompt the user to enter three integers.
- c) Define the variables x, y and z to be of type int.
- d) Read three integers from the keyboard and store them in the variables x, y and z.
- e) Define the variable result, compute the product of the integers in the variables x, y and
- z, and use that product to initialize the variable result.
- f) Print "The product is" followed by the value of the integer variable result.

Homeworks

E3: Using the statements you wrote in E1, write a complete program that calculates the product of three integers.

E4: Identify and correct the errors in each of the following statements:

a)
```
printf( "The value is %d\n", &number );
```

b)
```
scanf( "%d%d", &number1, number2 );
```

c)
```
if ( c < 7 );{
    printf( "C is less than 7\n" );
}
```

d)
```
if ( c => 7 ) {
    printf( "C is greater than or equal to 7\n" );
}
```